

Αρχές Ορθού Προγραμματισμού

Άμεσα σχετιζόμενες με τις αρχές ορθού
σχεδιασμού και τεχνολογίας λογισμικού

Αρχές Ορθού Προγραμματισμού

- Συνοψίζουν καλές πρακτικές, η εφαρμογή των οποίων μπορεί να βοηθήσει
 - Τους Προγραμματιστές να γίνουν καλύτεροι, και
 - Τους Μηχανικούς Λογισμικού να είναι πιο αποτελεσματικοί και να παράγουν κώδικα που είναι ευκολότερος στη συντήρηση και με λιγότερα σφάλματα/προβλήματα
- Πάντοτε έχοντας υπόψη ότι σε τελική ανάλυση προσπαθούμε να συγκεράσουμε δύο αντίπαλες έννοιες, τη **λεπτομέρεια** και τη **διαύγεια/απλότητα**
 - Η εφαρμογή των αρχών αυτών βοηθάει στον επιδιωκόμενο συνδυασμό

Κατάλογος Ορθών Αρχών/Πρακτικών Προγραμματισμού

Δεν είναι εξαντλητικός.

Συνεχώς εξελίσσεται μέσω της
συσσωρευμένης εμπειρίας της διεθνούς
κοινότητας των προγραμματιστών

Εν πολλοίς, οι αρχές αποτελούν
«κοινή λογική»

Είναι μια δέσμη **αλληλένδετων** αρχών και
πρακτικών ορθού προγραμματισμού

Κατάλογος Ορθών Αρχών/ Καλών Πρακτικών Προγραμματισμού

1. **Abstraction Principle** – Αρχή της Αφαιρετικότητας
2. **DRY** (Don't repeat yourself) – Μην επαναλαμβάνεις τον εαυτό σου
3. **KISS** (Keep it simple, stupid!) – Κράτα το απλό!
4. **Do the simplest thing that could possibly work** - Κάνε το απλούστερο πράγμα που θα μπορούσε να δουλέψει
5. **Avoid creating a YAGNI** (You aren't going to need it) – Απέφευγε τη δημιουργία αχρείαστων
6. **Don't make me think** – Μην με κάνεις να σκέφτομαι
7. **Write code for the Maintainer** – Γράφε κώδικα για τον συντηρητή

Συνέχεια ορθών αρχών/ καλών πρακτικών

- 8. **Principle of least astonishment** – Αρχή του ελάχιστου ξαφνιάσματος
- 9. **Open/Closed Principle** – Αρχή του Ανοικτού/Κλειστού
- 10. **Single Responsibility Principle** – Αρχή της μιας ευθύνης
- 11. **Minimize Coupling** – Ελαχιστοποίησε τη σύζευξη
- 12. **Maximize Cohesion** – Μεγιστοποίησε τη συνοχή
- 13. **Separation of Concerns** – Διαχωρισμός ευθυνών
- 14. **Hide Implementation Details** – Να αποκρύβεις τις λεπτομέρειες υλοποίησης

Συνέχεια ορθών αρχών/ καλών πρακτικών

- 15. Law of Demeter (LoD) or Principle of Least Knowledge** – Αρχή του όσο λιγότερο γνωρίζεις τόσο το καλύτερο
- 16. Avoid Premature Optimization** – Απέφευγε προώρη βελτιστοποίηση
- 17. Code Reuse is Good** – Επαναχρησιμοποίηση κώδικα
- 18. Embrace Change** – Αγκαλιάστε την αλλαγή

1. Αρχή της Αφαιρετικότητας

Abstraction Principle

- Κορυφαία αρχή ορθού προγραμματισμού, η οποία εμπερικλείει αρκετές από τις υπόλοιπες αρχές, π.χ.
 - Μην επαναλαμβάνεις τον εαυτό σου
 - Κράτα το απλό
 - Να αποκρύβεις τις λεπτομέρειες
 - Επαναχρησιμοποίηση κώδικα
- Αφορά τόσο τον κώδικα per se (program abstraction) όσο και τα δεδομένα (data abstraction)
- Βασικοί μηχανισμοί αφαιρετικότητας είναι
 - Η συνάρτηση/μέθοδος για program abstraction
 - Αρθρωτές λύσεις (modular solutions)
 - Αξιοποίηση Βιβλιοθηκών
 - Οι αφηρημένοι τύποι δεδομένων/αντικείμενα για data abstraction
 - Αξιοποίηση Κλάσεων Αντικειμένων

Αναφορές σε Αφαιρετικότητα

- Κάθε σημαντικό τμήμα λειτουργικότητας πρέπει να υλοποιείται μόνο μια φορά
- Όπου υπάρχουν ξεχωριστές, παρόμοιες συναρτήσεις, να αντικαθίστανται από μια πιο γενική συνάρτηση, με την αφαίρεση των διαφορών
- Σημαντικό κριτήριο ποιότητας για μια γλώσσα προγραμματισμού είναι πόσο καλά στηρίζει την αρχή της αφαιρετικότητας.

Προσπαθήστε να

- Σκέφτεστε όσο γίνεται πιο αφηρημένα/γενικά
- Γενικεύετε ένα πρόβλημα για να δείτε τις αναλογίες του με άλλα προβλήματα και τις λύσεις τους
- Εστιάζετε στην ουσία, αγνοώντας τη λεπτομέρεια, η οποία μπορεί να εισαχθεί σταδιακά στη συνέχεια, όταν και όπου χρειάζεται

2. Μην επαναλαμβάνεις τον εαυτό σου

DRY – Don't Repeat Yourself

- Εμπειρικλείεται στην αρχή της αφαιρετικότητας
 - Μόλις αντιληφθείς ότι επαναλαμβάνεις τον εαυτό σου, δημιούργησε μια νέα αφαιρετικότητα
- Άμεσα σχετιζόμενη με την αλγοριθμική δομή της επανάληψης και τις έννοιες της συνάρτησης/μεθόδου και της κλάσης
 - Και αυτό επειδή μας επιτρέπουν στο να μην επαναλαμβάνουμε τον εαυτό μας
 - Π.χ. έχουμε να ταξινομήσουμε με τον ίδιο τρόπο δύο ξεχωριστούς πίνακες. Αντί να γράφουμε τον ίδιο κώδικα δύο φορές, τον ενσωματώνουμε σε μια συνάρτηση την οποία καλούμε όσες φορές θέλουμε.

3. Κράτα το απλό

KISS – Keep It Simple, Stupid!

- Κλασσική αρχή που δεν αφορά μόνο τον προγραμματισμό, αφού τα περισσότερα συστήματα λειτουργούν καλύτερα αν είναι απλά παρά περίπλοκα
- Κατ' επέκταση η **απλότητα** πρέπει να συνιστά βασική αρχή σχεδιασμού
 - Simplicity is best
 - Unnecessary complexity should be avoided
- Ο κώδικας που είναι απλός απαιτεί λιγότερο χρόνο για να γραφτεί, έχει λιγότερα σφάλματα και διευκολύνει τροποποιήσεις
- Εξ ορισμού η αφηρημένη σκέψη είναι πιο απλή, αφού αφαιρεί/κρύβει λεπτομέρεια
- Σύνθετα προβλήματα διασπάζονται σε απλούστερα υπό-προβλήματα

4. Κάνε το απλούστερο πράγμα που θα μπορούσε να δουλέψει

Do the simplest thing that could possibly work

- Μια καλή ερώτηση προς τον εαυτό σας όταν προγραμματίζετε είναι
 - «Τι είναι το απλούστερο πράγμα το οποίο θα μπορούσε ενδεχομένως να δουλέψει;»
 - Αυτή η ερώτηση σας βάζει στο σωστό δρόμο για την επίτευξη απλότητας στο σχεδιασμό
- Ενδεχομένως αρχίζετε με κάτι που «δουλεύει» και σταδιακά το αντικαθιστάτε με κάτι απλούστερο που επίσης «δουλεύει»
- Η απλότητα σαφώς και είναι κριτήριο «ποιοτικού» σχεδιασμού
 - Σημ.: Άλλα κριτήρια απόδοσης, π.χ. ταχύτητα/μνήμη μπορεί να συνηγορούν προς λύσεις που δεν είναι κατ' ανάγκη οι πιο απλές

Μια καλή τεχνική ...

- Που (έμμεσα) εφαρμόζει την αρχή της απλότητας και που μπορεί να οδηγήσει στην πιο απλή λύση για δεδομένο πρόβλημα είναι
 - Πρώτα επιλύουμε μια πιο απλή εκδοχή του προβλήματος, και
 - Μετά επεκτείνουμε τη λύση μας για να καλύψει το κανονικό πρόβλημα
 - Η επιτυχία/ευκολία της επέκτασης εξαρτάται σε σημαντικό βαθμό από την καταλληλότητα της άρθρωσης της λύσης για την πιο απλή εκδοχή του προβλήματος
 - Ποιο υπό-πρόβλημα χρειάζεται να επεκταθεί

5. Απέφευγε τη δημιουργία αχρείαστων

Avoid creating a YAGNI

- Άμεσα συναφής με την αρχή της απλότητας (KISS) και του να επιδιώκεται η πιο απλή λύση που «δουλεύει»
- Το τι σας λέει η αρχή αυτή είναι να μην προσθέτετε λειτουργικότητα πριν τη χρειάζεστε!
- Τυχόν «πλεονασμός» μειώνει την απλότητα και δυσκολεύει την κατανόηση του κώδικα, εύλογα δημιουργώντας απορίες ως προς τη (λογική της) παρουσία «αχρείαστων» στοιχείων
- Always implement things when you actually need them, never when you just foresee that you need them

6. Μην με κάνεις να σκέφτομαι

Don't make me think

- Ακόμη μια αρχή ορθού προγραμματισμού, άμεσα σχετιζόμενη με την αρχή της απλότητας, αφού το τι πρεσβεύει είναι ότι ο κώδικας θα πρέπει να μπορεί εύκολα να διαβαστεί και να γίνει κατανοητός με ελάχιστη προσπάθεια
 - Εάν κάποιος κώδικας απαιτεί μεγάλη σκέψη από ένα ανεξάρτητο παρατηρητή για να γίνει κατανοητός, τότε μάλλον χρειάζεται απλοποίηση (ενδεχομένως μέσω καλύτερου σχεδιασμού και άρθρωσης – μικρότερα τεμάχια εξ ορισμού είναι πιο εύκολα στην κατανόηση)

7. Γράφε κώδικα για τον Συντηρητή

Write code for the Maintainer

- Σχεδόν κάθε κώδικας που αξίζει τον κόπο να γραφτεί, αξίζει και τον κόπο να συντηρείται στο μέλλον, είτε από εσάς που τον γράψατε, είτε από κάποιους άλλους
 - Ακόμη και ο δικός σας κώδικας, μελλοντικά μπορεί να σας φαίνεται ότι έχει γραφτεί από κάποιο άλλο
 - Επομένως είναι καλύτερα όταν γράφετε κώδικα να θεωρείτε ότι αυτός θα συντηρείται από κάποιο άλλο
 - Ένας τρόπος για να μη ξεχνάτε ποτέ αυτή την αρχή είναι ο εξής:
 - “Always code as if the person who ends up maintaining your code is a violent psychopath who knows where you live.”

Να έχετε υπόψη ότι ...

- Η συντήρηση είναι η πιο δαπανηρή φάση οποιουδήποτε έργου που αφορά λογισμικό
- Καθαρός και κατανοητός κώδικας μειώνει το κόστος της συντήρησής του, επειδή
 - Διευκολύνει τυχόν τροποποιήσεις του
 - Μειώνει την πιθανότητα εισδοχής σφαλμάτων από μέρους των συντηρητών
- Προσπαθήστε να γράφετε κώδικα
 - Που είναι εύκολος στις αλλαγές, και
 - Που η όποια αλλαγή έχει περιορισμένη επίδραση
 - Οι συνέπειες της είναι αντιληπτές/προβλέψιμες και επομένως διαχειρίσιμες

Να επιδιώκετε ...

- Να γράφετε κώδικα και σχόλια σε αυτόν, με τέτοιο τρόπο ώστε άλλοι προγραμματιστές, κυρίως σε χαμηλότερη βαθμίδα από εσάς, να αντλούν ευχαρίστηση από το να διαβάζουν τον κώδικά σας και να μαθαίνουν από αυτόν! Και τα Μεγάλα Γλωσσικά Μοντέλα.
 - Μεγάλη ηθική ικανοποίηση για έναν προγραμματιστή!
 - Δηλαδή, να εμπνέει/διδάσκει άλλους προγραμματιστές.
 - Ποιοτικοί/έξυπνοι αλγόριθμοι, ποιοτικά διατυπωμένοι σε κώδικα

8. Αρχή του Ελάχιστου Ξαφνιάσματος

Principle of least astonishment

- Αφορά πρωτίστως τη διασύνδεση του προγράμματος με το χρήστη, αλλά και το πρόγραμμα στην ολότητά του
 - Ο αναγνώστης του κώδικα πρέπει να ξαφνιάζεται όσο γίνεται πιο λίγο
 - Η παρεχόμενη λειτουργικότητα να ανταποκρίνεται στο τι υποδηλώνεται από το όνομα του προγράμματος και τα σχόλια που το συνοδεύουν
 - Παρενέργειες που δυνητικά μπορούν να προκαλέσουν έκπληξη πρέπει να αποφεύγονται όσο το δυνατόν
 - Εν ολίγοις το πρόγραμμα πρέπει να συμπεριφέρεται όπως ο χρήστης αναμένει από αυτό

9. Αρχή του Ανοικτού/Κλειστού

Open/Closed Principle

- Οι κλάσεις, ενότητες, συναρτήσεις, κτλ., να είναι **ανοικτές για επέκταση**, αλλά **κλειστές για τροποποιήσεις**
 - Με άλλα λόγια να κατασκευάζετε κλάσεις που οι άλλοι μπορούν να επεκτείνουν, αλλά όχι να αλλάζουν.
 - Με τον τρόπο αυτό δεν επηρεάζονται προγράμματα που ήδη αξιοποιούν κλάσεις στην υφιστάμενη μορφή τους. Οι επεκτάσεις είναι κάτι το επιπρόσθετο.
 - Η Java παρέχει διάφορους μηχανισμούς προς ικανοποίηση της αρχής του «ανοικτού/κλειστού»
 - Προσθήκη νέων πεδίων (αντικειμένων) ή/και μεθόδων (π.χ. overloading)
 - Κληρονόμηση, abstract base classes, polymorphism

Δομημένα, αρθρωτά προγράμματα

- Διευκολύνουν
 - τόσο τις **επεκτάσεις** τους, χωρίς επηρεασμό των υφιστάμενων τμημάτων τους
 - όσο και τις **αλλαγές** σε υφιστάμενα τμήματά τους, με απόλυτα διαχειρίσιμο τρόπο
- Συνεπώς διευκολύνουν την **ενίσχυση της λειτουργικότητας** που παρέχεται από αυτά

10 Αρχή της μίας ευθύνης

Single Responsibility Principle

- Κάθε κλάση ή συνάρτηση να εκτελεί μια μοναδική και σαφώς ορισμένη εργασία
- Με τον τρόπο αυτό μια κλάση έχει μια και μόνο ευθύνη
 - ευθύνεται δηλαδή για την παροχή μιας και μόνο λειτουργικότητας, και
 - αυτό την καθιστά πιο ανθεκτική, αφού
 - η ανάγκη για αλλαγή περιορίζεται σε ένα μόνο λόγο
- Άμεση συνάφεια με την αρχή της απλότητας
 - Απλοποιεί τη διασύνδεση της κλάσης και επομένως
 - Περιορίζει το περιεχόμενο της σε ένα μικρό αριθμό συναφών λογισμικών στοιχείων

11. Ελαχιστοποίησε τη σύζευξη

Minimize Coupling

- **Σύζευξη** ορίζεται ως ο βαθμός αλληλεξάρτησης μεταξύ μονάδων λογισμικού, με άλλα λόγια πόσο στενά συνδεδεμένες είναι δύο μονάδες λογισμικού, π.χ. δύο συναρτήσεις
 - Ισχυρή σύζευξη (υψηλός βαθμός συνδεσιμότητας) μειώνει την αυτοδυναμία των εμπλεκόμενων μονάδων
 - Ισχυρή σύζευξη υπάρχει όταν για παράδειγμα συναρτήσεις μοιράζονται μεταξύ τους μεταβλητές
 - Ενδείκνυται η χρήση αποκλειστικά τοπικών μεταβλητών
 - Κατ' επέκταση η χρήση καθολικών μεταβλητών πρέπει να είναι άκρως περιορισμένη και απόλυτα δικαιολογημένη

Μειονεκτήματα ισχυρής σύζευξης

- Αλλαγές σε μια μονάδα, αλυσιδωτά ενδέχεται να απαιτούν αλλαγές σε άλλες μονάδες
- Η συναρμολόγηση των μονάδων απαιτεί μεγαλύτερη προσπάθεια και χρόνο ως αποτέλεσμα των αλληλεξαρτήσεων ανάμεσα τους
- Δυσκολεύεται η δοκιμή και επαναχρησιμοποίηση μιας μονάδας αφού πρέπει να ληφθούν υπόψη και οι εξαρτήσεις της με άλλες μονάδες
 - Με απλά λόγια δεν υπάρχει αυτοδυναμία και ανεξαρτησία για την κάθε μονάδα ξεχωριστά

Συνεπώς, ελαχιστοποίησε τη σύζευξη
*Μειώνοντας τις αλληλεξαρτήσεις ανάμεσα
στα τμήματα του κώδικα σου*

- Αδύνατη/χαλαρή σύζευξη είναι χαρακτηριστικό ορθού σχεδιασμού και ορθά δομημένου κώδικα
- Και, σε συνδυασμό με ισχυρή συνοχή, ενισχύει την αναγνωσιμότητα και συντηρησιμότητα του κώδικα

12. Μεγιστοποίησε τη συνοχή

Maximize Cohesion

- **Συνοχή** είναι ο βαθμός της συνάφειας/ σχετικότητας των στοιχείων που απαρτίζουν μια μονάδα λογισμικού
 - Με άλλα λόγια ανήκουν μαζί οι μέθοδοι και τα δεδομένα που απαρτίζουν μια κλάση;
 - Η κλάση εξυπηρετεί κάποιο κοινό σκοπό ή έννοια;
- Ισχυρή συνοχή συνδέεται με ανθεκτικότητα, αξιοπιστία, επαναχρησιμοποίηση, και κατανόηση, του κώδικα
 - Σε αντίθεση, αδύνατη συνοχή συνδέεται με δυσκολίες στη δοκιμή, συντήρηση, επαναχρησιμοποίηση ή κατανόηση

Με απλά λόγια

- Κώδικες με συναφή λειτουργικότητα, θα πρέπει κανονικά να ανήκουν μαζί στην ίδια μονάδα
 - Π.χ. Βιβλιοθήκες
- Κατ' επέκταση η συστέγαση στην ίδια κλάση άσχετων μεταξύ τους τμημάτων, οδηγεί σε μια δυσνόητη κλάση χωρίς συγκεκριμένο σκοπό/λειτουργικότητα που θα οδηγηθεί σε αχρηστία

Θεωρητικά ...

- Μία μονάδα που αποτελείται από ένα και μόνο στοιχείο έχει «τέλεια» συνοχή
- Στην πραγματικότητα δεν είναι έτσι
 - Πρέπει να υπάρχει ένα ορθό ισοζύγιο ανάμεσα σε συνοχή, πολυπλοκότητα ατομικών στοιχείων και σύζευξη
 - Μοναδιαία στοιχεία είναι είτε πολύ περίπλοκα, είτε πολύ περιορισμένα και με ισχυρές συζεύξεις με άλλα (εξωγενή) στοιχεία

13. Διαχωρισμός ευθυνών

Separation of concerns

- Διαφορετικές λειτουργικότητες πρέπει να υλοποιούνται σε ξεχωριστά και με ελάχιστες επικαλύψεις, τμήματα κώδικα
 - Το καθένα με τη δική του ευθύνη (βλ. **αρχή της μιας ευθύνης**)
- Η εφαρμογή της αρχής του διαχωρισμού ευθυνών
 - οδηγεί σε αρθρωτά προγράμματα
 - απλοποιεί την ανάπτυξη και συντήρηση προγραμμάτων
 - διευκολύνει την επαναχρησιμοποίηση, και ανεξάρτητη ανάπτυξη/τροποποίηση τμημάτων του κώδικα

Είναι μια έκφραση της γενικής στρατηγικής ..

- Το πρόβλημα/στόχος διασπάζεται σε απλούστερα τμήματα, το καθένα με το δικό του σαφή υπο-στόχο (ευθύνη)
 - Διαίρει και βασίλευε
- Η διάσπαση που στοχεύει στο διαχωρισμό ευθυνών μπορεί να οργανωθεί σε διάφορα στρώματα αφαιρετικότητας
 - σχεδιασμός σε στρώσεις (layered design)

Έχει άμεση συνάφεια με την ...

- Αρχή της απόκρυψης πληροφοριών (λεπτομερειών υλοποίησης)
 - αφού η ενθυλάκωση (encapsulation) είναι ένας τρόπος επίτευξης άρθρωσης και διαχωρισμού ευθυνών
 - Όπου ορίζονται σαφείς διεπαφές/διαπροσωπείες (interfaces)
 - Τμήματα του κώδικα μπορούν να τροποποιούνται και να βελτιώνονται ξεχωριστά χωρίς να απαιτείται
 - γνώση των λεπτομερειών άλλων τμημάτων
 - αλλαγή σε άλλα τμήματα

Η Java παρέχει πολλούς κατασκευαστικούς μηχανισμούς για την επίτευξη διαχωρισμού ευθυνών

- Κλάσεις
- Συναρτήσεις/μεθόδους
- Αντικείμενα
 - ενθυλάκωση
- Που βεβαίως δεν είναι άλλοι από τους τρόπους στήριξης της κορυφαίας αρχής της αφαιρετικότητας

14. Να αποκρύβεις τις λεπτομέρειες υλοποίησης

Hide Implementation Details

- Αποκρύβοντας τις λεπτομέρειες υλοποίησης, μπορείς να αλλάξεις την υλοποίηση κάποιας μονάδας του κώδικα, επηρεάζοντας στο ελάχιστο άλλες μονάδες που την χρησιμοποιούν
 - Διαχωρισμός ανάμεσα στο **τι** (λειτουργικότητα παρέχεται) και το **πώς** (υλοποιείται η λειτουργικότητα)
 - Ξεκάθαρες και σταθερές διεπαφές/διαπροσωπείες, π.χ. Βιβλιοθήκες
 - Εάν ο ρόλος κάποιας μονάδας είναι καθαρά βοηθητικός προς άλλη/ες μονάδα/ες, η ύπαρξή της δεν χρειάζεται να είναι γνωστή (**private**)
 - Αντικείμενα
 - μόνο μέθοδοι αναφοράς public – εσωτερική κατάσταση private - **ενθυλάκωση**

Και η απόκρυψη πληροφοριών

- Είναι άμεσα σχετιζόμενη με την κορυφαία αρχή της αφαιρετικότητας
- **Αφηρημένοι Τύποι Δεδομένων** (ADT – Abstract Data types) - Αντικείμενα
 - Η δομή υλοποίησης αποκρύβεται
 - Μόνο οι μέθοδοι πρόσβασης (access functions) δημοσιοποιούνται – για τη δημιουργία περιπτώσεων, την ανάκληση στοιχείων και την αλλαγή στοιχείων
 - Ουσιαστικά οι διεπαφές τους

15. Αρχή του όσο λιγότερο γνωρίζεις τόσο το καλύτερο

Principle of Least Knowledge - LoD

- Σχετίζεται με την αρχή της απόκρυψης πληροφοριών
- Επιπρόσθετα μπορεί να ιδωθεί και ως μια ειδική περίπτωση «χαλαρής σύζευξης» (loose coupling)
 - Αυτοδυναμία τμημάτων κώδικα
- Αναφέρεται και ως ο «Νόμος της Δήμητρας»
 - Διατυπώθηκε στο πλαίσιο του έργου “Demeter” προς τιμή της Δήμητρας, της θεάς της γεωργίας
 - “distribution-mother”
 - φιλοσοφία προγραμματισμού «από κάτω προς τα πάνω»

Αφορά κυρίως τον αντικειμενοστρεφή προγραμματισμό

- “Law of Demeter for Functions/Methods”: LoD–F
- Κάθε μονάδα να έχει περιορισμένη γνώση των υπολοίπων μονάδων
 - Να επικοινωνεί μόνο με αυτές με τις οποίες έχει άμεση σχέση
 - Κλάσεις από τις οποίες κληρονομεί
 - Αντικείμενα που περιλαμβάνει ή που λαμβάνει ως ορίσματα
- Γενικά ένα αντικείμενο να γνωρίζει όσο γίνεται πιο λίγα για τη δομή και τις ιδιότητες άλλων αντικειμένων
 - Εσωτερικές δομές αντικειμένων μπορεί να αλλάζουν χωρίς να επηρεάζουν άλλα αντικείμενα
 - Ο κώδικας είναι πιο συντηρήσιμος και προσαρμόσιμος

Η πολύ-επίπεδη αρχιτεκτονική είναι ...

- Ένας τρόπος υλοποίησης της αρχής του όσο το λιγότερο γνωρίζεις, τόσο το καλύτερο
- Οι κλήσεις (σε συναρτήσεις) περιορίζονται εντός του ιδίου στρώματος ή του αμέσως πιο κάτω στρώματος

16. Απέφυγε πρόωρη βελτιστοποίηση

Avoid premature optimization

- **Donald Knuth:** *“We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil.”*
- Πριν να έχεις ένα ορθό πρόγραμμα, δεν πρέπει να σκέφτεσαι για βελτιστοποίηση
 - Η διεργασία της «βελτιστοποίησης» δεν οδηγεί κατ’ ανάγκη σε «βέλτιστο» πρόγραμμα
 - Τι σημαίνει «βέλτιστο»;
 - Πιο γρήγορο;
 - Που αξιοποιεί λιγότερη μνήμη;
 - Πρέπει να τεθούν τα σχετικά κριτήρια «βελτιστοποίησης»

Πιο ορθά να μιλούμε για ποιοτική βελτίωση

- Βάσει των αρχών ορθού προγραμματισμού
 - Σε ποια ποιοτικά κριτήρια αποδίδεται μεγαλύτερη βαρύτητα;
- Σε μια ιεραρχική δομή:
 - τα υψηλότερα επίπεδα (αφαιρετικότητα) έχουν μεγαλύτερη επίδραση αφού καθορίζουν την ουσία του σχεδιασμού
 - είναι δύσκολο να τροποποιηθούν στη συνέχεια
 - μπορεί να απαιτείται ολικός ανασχεδιασμός
 - τυχόν βελτιώσεις σε σχέση με το χρόνο εκτέλεσης ή/και την αξιοποίηση μνήμης συνήθως συνεπάγονται τροποποιήσεις στα χαμηλότερα επίπεδα
 - Τέτοιες αλλαγές είναι τοπικές και μπορεί να είναι και αρκετά απλές, αλλά να έχουν σημαντική, καθολική, επίδραση

17. Επαναχρησιμοποίηση Κώδικα

Code reuse is good

- Η επαναχρησιμοποίηση κώδικα βελτιώνει την αξιοπιστία και μειώνει το χρόνο για την ανάπτυξη ενός νέου προγράμματος
- Λαμπρά παραδείγματα επαναχρησιμοποίησης αποτελούν οι **Βιβλιοθήκες**
- Η **άρθρωση** βοηθά την επαναχρησιμοποίηση
 - Δεν είθισται να επαναχρησιμοποιείται πρόγραμμα στην ολότητά του (cf συναρτήσεις `main` δεν επαναχρησιμοποιούνται)
- Καθώς επίσης και η χαλαρή σύζευξη, η ισχυρή συνοχή, η απόκρυψη πληροφοριών και ο διαχωρισμός ευθυνών

18. Αγκαλιάστε την αλλαγή

Embrace change

- Η ανάγκη για αλλαγές στον κώδικά σας, για διάφορους λόγους, συνιστά αναπόφευκτο φαινόμενο
 - Οι αλλαγές, επομένως, πρέπει να είναι αναμενόμενες και «καλοδεχούμενες»
- Οι πλείστες από τις αρχές ορθού προγραμματισμού που συζητήσαμε, π.χ. η ελαχιστοποίηση της σύζευξης, διευκολύνουν τις αλλαγές
 - Οι αλλαγές γίνονται με ελεγχόμενο τρόπο, με άλλα λόγια οι επιδράσεις τους είναι προβλέψιμες και διαχειρίσιμες
 - Μια αλλαγή, αλυσιδωτά οδηγεί σε άλλες (προβλέψιμες) αλλαγές